

SMTP Email Traffic Forensics

OJE DOMINION MAYOWA

FWSD2511507

Computer and Digital Forensics

MR. Aminu Idris

12 September 2026

LAB OVERVIEW

This lab focused on analyzing a captured SMTP session to reconstruct an email exchange from raw packet data. I identified the client-server conversation, extracted SMTP commands and response codes, decoded base64-encoded authentication credentials, and reconstructed the full email message including headers, body type, and attachment presence. I also examined network-layer metadata and confirmed that the session used no encryption, exposing credentials and content in plaintext despite STARTTLS being available.

OBJECTIVES

- Explain the role of SMTP and distinguish plaintext SMTP, STARTTLS and implicit TLS.
- Identify SMTP commands, server response codes and authentication exchanges.
- Reassemble an SMTP TCP stream and reconstruct message headers and body.
- Decode Base64 values in a controlled offline manner.
- Extract timestamps, client software, IP addresses, ports and MAC addresses.
- Assess evidential limitations when encryption is used or capture is incomplete.\

TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- Python 3 base64 module
- Md5sum and sha256sum
- Apache2
- Firefox, Chrome or Edge
- Wireshark, tshark

METHODOLOGY

Lab Folder Structure and Evidence Preparation

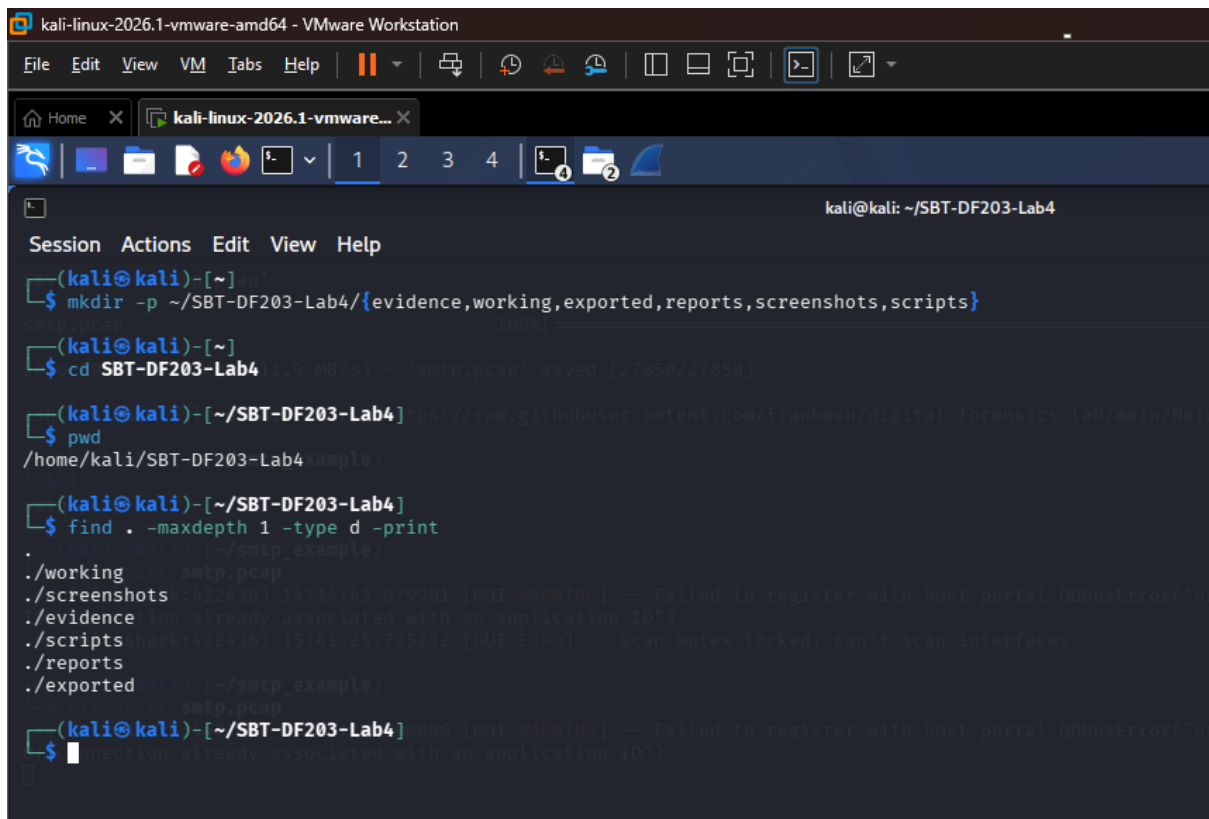


Fig 1: Creating a working environment for the lab work

Then we download the necessary tools needed for the lab-work and download the source evidence needed for the lab.

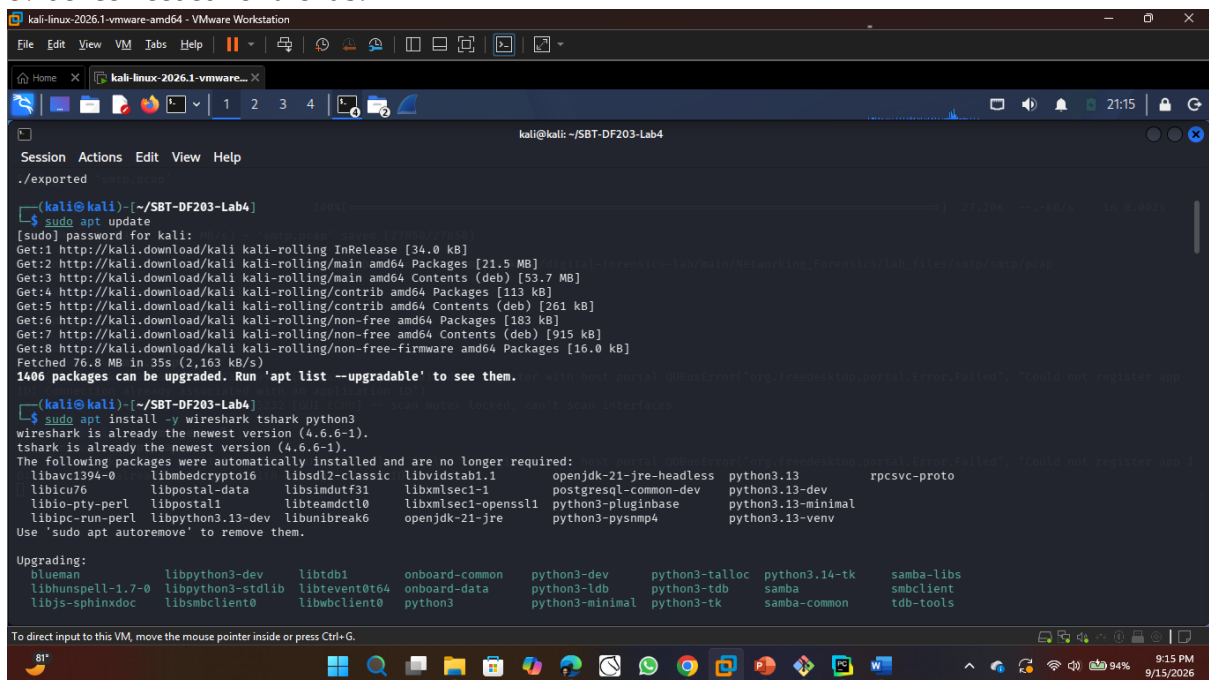


Fig 2:

Then we download the evidence from the source from online into the evidence folder

```
(kali@kali)-[~/SBT-DF203-Lab4]
$ wget -O evidence/smtp.pcap 'https://wiki.wireshark.org/uploads/_moin_import_/attachments/SampleCaptures/smtp.pcap'
--2026-09-15 21:20:03-- https://wiki.wireshark.org/uploads/_moin_import_/attachments/SampleCaptures/smtp.pcap
Resolving wiki.wireshark.org (wiki.wireshark.org)... 104.26.11.240, 172.67.75.39, 104.26.10.240, ...
Connecting to wiki.wireshark.org (wiki.wireshark.org)|104.26.11.240|:443 ... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://gitlab.com/wireshark/wireshark/-/wikis/uploads/_moin_import_/attachments/SampleCaptures/smtp.pcap [following]
--2026-09-15 21:20:05-- https://gitlab.com/wireshark/wireshark/-/wikis/uploads/_moin_import_/attachments/SampleCaptures/smtp.pcap
Resolving gitlab.com (gitlab.com)... 172.65.251.78, 2606:4700:90:0:f22e:fbec:5bed:a9b9
Connecting to gitlab.com (gitlab.com)|172.65.251.78|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 27850 (27K) [application/octet-stream]
Saving to: 'evidence/smtp.pcap'

evidence/smtp.pcap 100%[=====] 27.20K --.-KB/s in 0.01s

2026-09-15 21:20:06 (2.63 MB/s) - 'evidence/smtp.pcap' saved [27850/27850]

(kali@kali)-[~/SBT-DF203-Lab4]
```

Fig 3: Evidence download

Then we need to create a copy of the original evidence file

```
(kali@kali)-[~/SBT-DF203-Lab4]
$ cp --preserve=timestamps evidence/smtp.pcap working/smtp_working.pcap
```

Fig 4: Creating a Copy of the Original Evidence to keep the Integrity

Then we check for the integrity of the original source of the evidence and the copy of the original.

```
(kali@kali)-[~/SBT-DF203-Lab4]
$ sha256sum evidence/smtp.pcap working/smtp_working.pcap | tee reports/smtp_capture_hashes.txt
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab evidence/smtp.pcap
17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab working/smtp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab4]
```

Fig 5: Checking the Integrity of both the original and Copy through the hash values

Then we get metadata details of the evidence file by using the **capinfos** command and then we put it into a reporting file **smtp_capinfos.txt**

```

kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.1-vmware... X
kali@kali: ~/SBT-DF203-Lab4
Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab4]
$ capinfos evidence/smtp.pcap | tee reports/smtp_capinfos.txt
File name:          evidence/smtp.pcap
File type:          Wireshark/tcpdump/... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit:  file hdr: 65535 bytes
Number of packets:  60
File size:          27 kB
Data size:          26 kB
Capture duration:   9.198384 seconds
Earliest packet time: 2009-10-05 07:06:07.492060
Latest packet time:  2009-10-05 07:06:16.690444
Data byte rate:      2,920 bytes/s
Data bit rate:        23 kbps
Average packet size: 447.77 bytes
Average packet rate:  6 packets/s
SHA256:              17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab
SHA1:                3def1a1fed849e7f23b66e6925ddff05845a400b
Strict time order:   True
Number of interfaces in file: 1
Interface #0 info:
  Encapsulation = Ethernet (1 - ether)
  Capture length = 65535
  Time precision = microseconds (6)
  Time ticks per second = 1000000
  Number of stat entries = 0
  Number of packets = 60
(kali@kali)-[~/SBT-DF203-Lab4]
$

```

Fig 6: Metadata Details of the Evidence File

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab4-Oje Dominion Mayowa
Trainee name	OJE DOMINION MAYOWA
Date and time started	15/09/2026, 9:00AM
Evidence file name(s)	smtp.pcap
Source or generation method	Downloaded from a github link
Original SHA-256	17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab
Working-copy SHA-256	17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab

Field	Student Entry
Analysis workstation/VM	Kali Linux
Notes on any changes	

Part A - Inventory the Capture and Locate SMTP Streams

The first step is to capture and locate smtp streams from the server

```
(kali@kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap -q -z conv,tcp | tee reports/tcp_conversations.txt

TCP Conversations
Filter:<No Filter>

| Frames | Bytes | | Frames | Bytes | | Total | Relative | Duration |
|-----|-----| |-----|-----| |-----|-----|-----|
10.10.1.4:1470 ↔ 74.53.140.153:25 | 25 | 1,980 bytes | 28 | 22 kB | 53 | 24 kB | 0.036986000 | 7.5777
```

Fig 7:

```
(kali@kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap -Y 'smtp' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e _ws.col.Info \
  | tee reports/smtp_packet_inventory.tsv

6      2009-10-05T07:06:08.219663000+0100    74.53.140.153    25    10.10.1.4    1470    S: 220-xc90.websitewelcome.com ESMTP Exim 4.69 #1 Mon, 05 Oct 2009 01:
05:54 -0500 | We do not authorize the use of this system to transport unsolicited, and/or bulk e-mail.
7      2009-10-05T07:06:08.224809000+0100    10.10.1.4      1470    74.53.140.153    25    C: EHLO GP
9      2009-10-05T07:06:08.566183000+0100    74.53.140.153    25    10.10.1.4      1470    S: 250-xc90.websitewelcome.com Hello GP [122.162.143.157] | SIZE 52428
800 | PIPELINING | AUTH PLAIN LOGIN | STARTTLS | HELP
10     2009-10-05T07:06:08.568729000+0100    10.10.1.4      1470    74.53.140.153    25    C: AUTH LOGIN
11     2009-10-05T07:06:08.911081000+0100    74.53.140.153    25    10.10.1.4      1470    S: 334 Username:
12     2009-10-05T07:06:08.911655000+0100    10.10.1.4      1470    74.53.140.153    25    C: User: gurpartap@patriots.in
13     2009-10-05T07:06:09.253544000+0100    74.53.140.153    25    10.10.1.4      1470    S: 334 Password:
14     2009-10-05T07:06:09.254118000+0100    10.10.1.4      1470    74.53.140.153    25    C: Pass: punjab@123
15     2009-10-05T07:06:09.613798000+0100    74.53.140.153    25    10.10.1.4      1470    S: 235 Authentication succeeded
16     2009-10-05T07:06:09.614414000+0100    10.10.1.4      1470    74.53.140.153    25    C: MAIL FROM: <gurpartap@patriots.in>
17     2009-10-05T07:06:09.956765000+0100    74.53.140.153    25    10.10.1.4      1470    S: 250 OK
18     2009-10-05T07:06:09.957250000+0100    10.10.1.4      1470    74.53.140.153    25    C: RCPT TO: <raj_deol2002in@yahoo.co.in>
19     2009-10-05T07:06:10.319708000+0100    74.53.140.153    25    10.10.1.4      1470    S: 250 Accepted
20     2009-10-05T07:06:10.320203000+0100    10.10.1.4      1470    74.53.140.153    25    C: DATA
21     2009-10-05T07:06:10.661679000+0100    74.53.140.153    25    10.10.1.4      1470    S: 354 Enter message, ending with "." on a line by itself
22     2009-10-05T07:06:10.692743000+0100    10.10.1.4      1470    74.53.140.153    25    C: DATA fragment, 1460 bytes
23     2009-10-05T07:06:10.692786000+0100    10.10.1.4      1470    74.53.140.153    25    C: DATA fragment, 1460 bytes
24     2009-10-05T07:06:10.692804000+0100    10.10.1.4      1470    74.53.140.153    25    C: DATA fragment, 1460 bytes
25     2009-10-05T07:06:10.692823000+0100    10.10.1.4      1470    74.53.140.153    25    [TCP Window Full] C: DATA fragment, 1460 bytes
26     2009-10-05T07:06:10.695115000+0100    192.168.1.1,10.10.1.4    1470    10.10.1.4,74.53.140.153    25    Destination unreachable (Fragmentation needed)
28     2009-10-05T07:06:10.695623000+0100    192.168.1.1,10.10.1.4    1470    10.10.1.4,74.53.140.153    25    Destination unreachable (Fragmentation needed)
29     2009-10-05T07:06:10.696248000+0100    192.168.1.1,10.10.1.4    1470    10.10.1.4,74.53.140.153    25    Destination unreachable (Fragmentation needed)
30     2009-10-05T07:06:10.696634000+0100    192.168.1.1,10.10.1.4    1470    10.10.1.4,74.53.140.153    25    Destination unreachable (Fragmentation needed)
32     2009-10-05T07:06:11.104972000+0100    10.10.1.4      1470    74.53.140.153    25    [TCP Out-Of-Order] C: DATA fragment, 1452 bytes
33     2009-10-05T07:06:11.104998000+0100    10.10.1.4      1470    74.53.140.153    25    [TCP Out-Of-Order] C: DATA fragment, 1452 bytes
35     2009-10-05T07:06:11.469759000+0100    10.10.1.4      1470    74.53.140.153    25    [TCP Out-Of-Order] C: DATA fragment, 1452 bytes
36     2009-10-05T07:06:11.469814000+0100    10.10.1.4      1470    74.53.140.153    25    [TCP Out-Of-Order] C: DATA fragment, 1452 bytes
38     2009-10-05T07:06:11.494181000+0100    10.10.1.4      1470    74.53.140.153    25    C: DATA fragment, 1452 bytes
39     2009-10-05T07:06:11.494199000+0100    10.10.1.4      1470    74.53.140.153    25    C: DATA fragment, 1452 bytes
41     2009-10-05T07:06:11.834628000+0100    10.10.1.4      1470    74.53.140.153    25    C: DATA fragment, 1452 bytes
42     2009-10-05T07:06:11.834655000+0100    10.10.1.4      1470    74.53.140.153    25    C: DATA fragment, 1452 bytes
44     2009-10-05T07:06:11.858316000+0100    10.10.1.4      1470    74.53.140.153    25    C: DATA fragment, 1452 bytes
```

Fig 8:

SMTP server port: 25 (standard SMTP port, on server 74.53.140.153)

Client ephemeral port: 1470 (on client 10.10.1.4)

Sequence	Direction	Command/Code	Meaning	Timestamp
1	Server to client	220	Service ready	2009-10-05T07:06:08.219663000+0100
2	Client to server	EHLO/HELO	Client greeting	2009-10-05T07:06:08.224809000+0100
3	Client to server	AUTH	Authentication method	2009-10-05T07:06:08.568729000+0100
4	Client to server	MAIL FROM	Envelope sender	2009-10-05T07:06:09.614414000+0100
5	Client to server	RCPT TO	Envelope recipient	2009-10-05T07:06:09.957250000+0100
6	Client to server	DATA	Message content begins	2009-10-05T07:06:10.320203000+0100
7	Client to server	QUIT	Session termination	2009-10-05T07:06:14.763825000+0100

Part C - Decode Base64 Authentication Evidence

Initially when I wanted to check the decoded base64 from the packets evidence collected I noticed that it has already been decoded into the plain text and then I had to go their raw form to be able to get the base64 format from their frame numbers

The Username: **Z3VycGFydGFwQHBhdHJpb3RzLmlu**

The password: **chVuamFiQDEyMw==**

```
(kali㉿kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap -Y 'frame.number==12' -x | tail -20
0000  00 1f 33 d9 81 60 00 e0 1c 3c 17 c2 08 00 45 00  ..3..`...<....E.
0010  00 46 25 29 40 00 80 06 f3 ac 0a 0a 01 04 4a 35  .F%)@.....J5
0020  8c 99 05 be 00 19 7e c4 53 c6 ae ec 63 00 50 18  .....~.S...c.P.
0030  fe af 22 a4 00 00 5a 33 56 79 63 47 46 79 64 47  .." ...Z3VycGFydG
0040  46 77 51 48 42 68 64 48 4a 70 62 33 52 7a 4c 6d  FwQHBhdHJpb3RzLm
0050  6c 75 0d 0a                                     lu..

(kali㉿kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap -Y 'frame.number==14' -x | tail -20
0000  00 1f 33 d9 81 60 00 e0 1c 3c 17 c2 08 00 45 00  ..3..`...<....E.
0010  00 3a 25 2e 40 00 80 06 f3 b3 0a 0a 01 04 4a 35  .:%.@.....J5
0020  8c 99 05 be 00 19 7e c4 53 e4 ae ec 63 12 50 18  .....~.S...c.P.
0030  fe 9d 54 b1 00 00 63 48 56 75 61 6d 46 69 51 44  ..T ...chVuamFiQD
0040  45 79 4d 77 3d 3d 0d 0a                         EyMw== ..

(kali㉿kali)-[~/SBT-DF203-Lab4]
$
```

Fig 10: Identifying the base64 values of the Username and the password

Now running a python script to be able to decode the base64 encoding which it did successfully

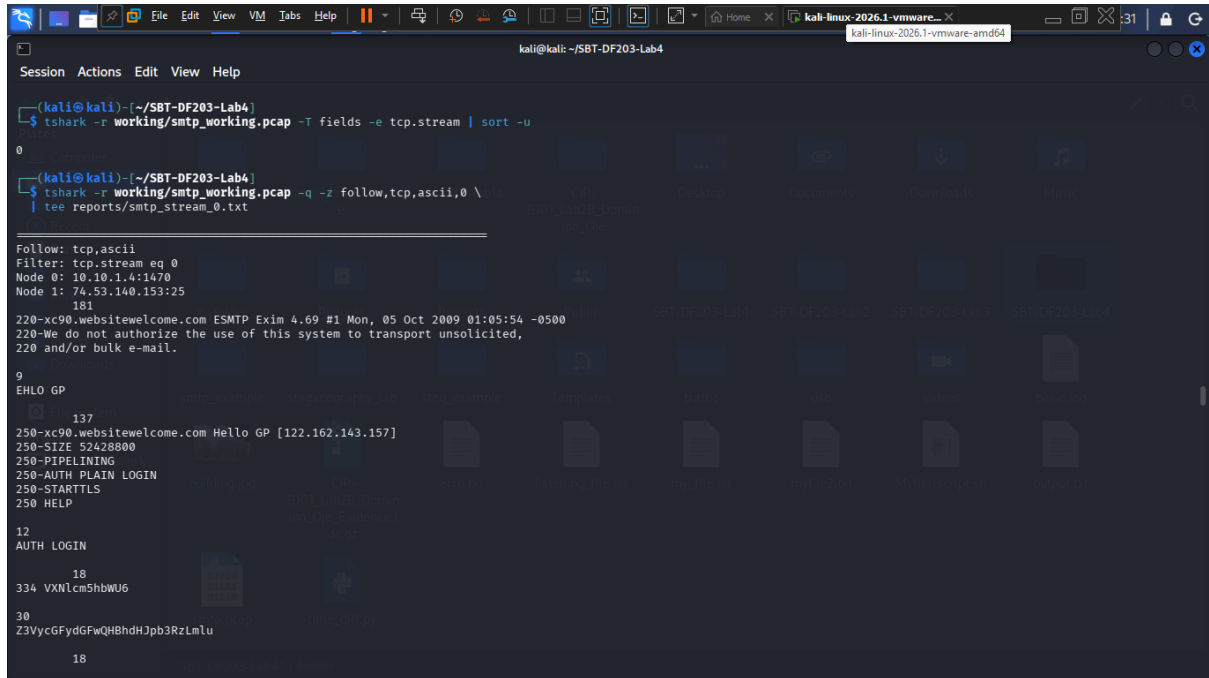
```
(kali㉿kali)-[~/SBT-DF203-Lab4]
$ python3 -c <<'PYLAB'
import base64
samples = {
    'username': 'Z3VycGFydGFwQHBhdHJpb3RzLmlu',
    'password': 'chVuamFiQDEyMw=='
}
for label, value in samples.items():
    try:
        decoded = base64.b64decode(value).decode('utf-8', errors='replace')
        print(f'{label}: {decoded}')
    except Exception as exc:
        print(f'{label}: decode failed: {exc}')
PYLAB
username: gурpartap@patriots.in
password: punjab@123

(kali㉿kali)-[~/SBT-DF203-Lab4]
$
```

Fig 11: A Script to decode base64

Part D - Reconstruct the Email Message

Getting to know the number of streams and then we reconstruct the email message



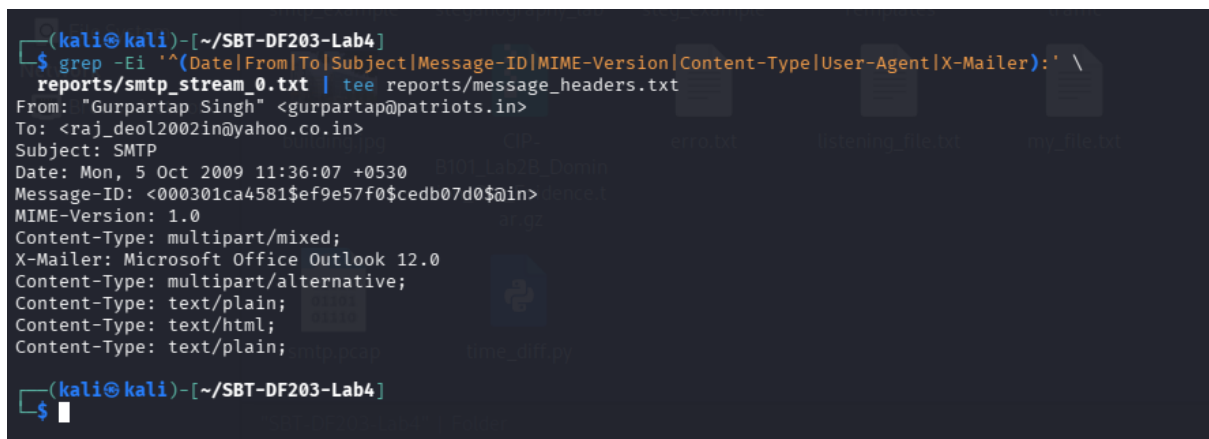
```
kali@kali: ~/SBT-DF203-Lab4
Session Actions Edit View Help

(kali@kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/sntp_working.pcap -T fields -e tcp.stream | sort -u
0

(kali@kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/sntp_working.pcap -q -z follow,tcp,ascii,0 \
  | tee reports/sntp_stream_0.txt

Follow: tcp,ascii
Filter: tcp.stream eq 0
Node 0: 10.10.1.4:1470
Node 1: 74.53.140.153:25
181
220-xc90.websitewelcome.com ESMTP Exim 4.69 #1 Mon, 05 Oct 2009 01:05:54 -0500
220-We do not authorize the use of this system to transport unsolicited,
220 and/or bulk e-mail.
9
EHLO GP
137
250-xc90.websitewelcome.com Hello GP [122.162.143.157]
250-SIZE 52428800
250-PIPELINING
250-AUTH PLAIN LOGIN
250-STARTTLS
250 HELP
12
AUTH LOGIN
18
334 VXNlcm5hbWU6
30
Z3VycGFydGFwQHBlb3RzLmLu
18
```

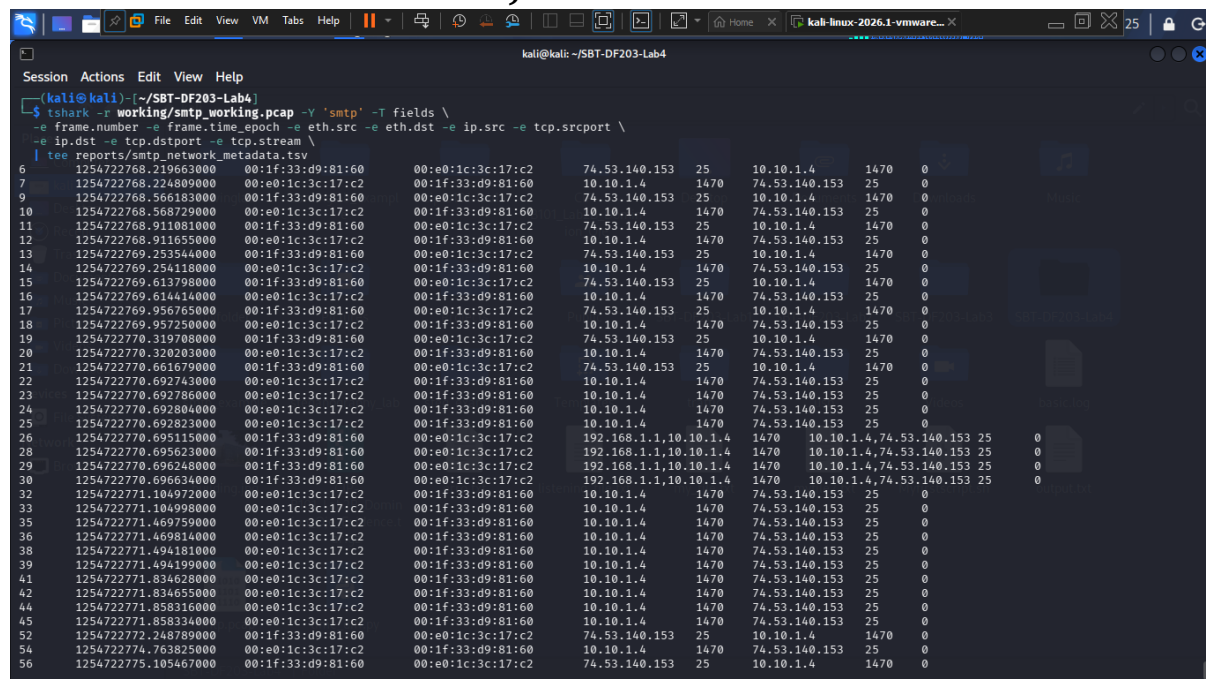
Fig 12: Reconstructing an Email Message



```
(kali@kali)-[~/SBT-DF203-Lab4]
$ grep -Ei '^(Date|From|To|Subject|Message-ID|MIME-Version|Content-Type|User-Agent|X-Mailer):' \
  reports/sntp_stream_0.txt | tee reports/message_headers.txt
From: "Gurpartap Singh" <gurpartap@patriots.in>
To: <raj_deol2002in@yahoo.co.in>
Subject: SMTP
Date: Mon, 5 Oct 2009 11:36:07 +0530
Message-ID: <000301ca4581$ef9e57f0$cedb07d0$@in>
MIME-Version: 1.0
Content-Type: multipart/mixed;
X-Mailer: Microsoft Office Outlook 12.0
Content-Type: multipart/alternative;
Content-Type: text/plain;
Content-Type: text/html;
Content-Type: text/plain;
```

Fig 13: Getting the required information from the reconstructed email message

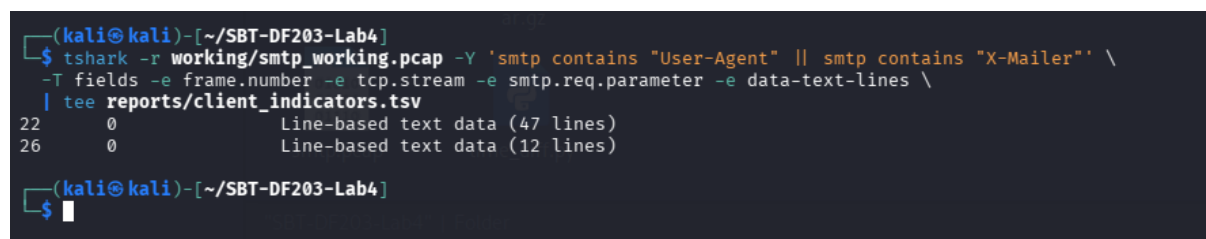
Part E - Determine Client, Hosts and Network Metadata



```
Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap -Y 'smtp' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e tcp.srcport \
  -e ip.dst -e tcp.dstport -e tcp.stream \
  | tee reports/smtp_network_metadata.tsv
6      1254722768.219663000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 74.53.140.153 25 10.10.1.4 1470 0
7      1254722768.224809000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
9      1254722768.566183000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 74.53.140.153 25 10.10.1.4 1470 0
10     1254722768.568729000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
11     1254722768.911081000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 74.53.140.153 25 10.10.1.4 1470 0
12     1254722768.911655000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
13     1254722769.253544000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 74.53.140.153 25 10.10.1.4 1470 0
14     1254722769.254118000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
15     1254722769.613798000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 74.53.140.153 25 10.10.1.4 1470 0
16     1254722769.614414000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
17     1254722769.956765000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 74.53.140.153 25 10.10.1.4 1470 0
18     1254722769.957250000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
19     1254722770.319708000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 74.53.140.153 25 10.10.1.4 1470 0
20     1254722770.320230000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
21     1254722770.662679000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 74.53.140.153 25 10.10.1.4 1470 0
22     1254722770.692743000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
23     1254722770.692786000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
24     1254722770.692804000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
25     1254722770.692823000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
26     1254722770.695115000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 192.168.1.1,10.10.1.4 1470 10.10.1.4,74.53.140.153 25 0
28     1254722770.695623000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 192.168.1.1,10.10.1.4 1470 10.10.1.4,74.53.140.153 25 0
29     1254722770.696248000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 192.168.1.1,10.10.1.4 1470 10.10.1.4,74.53.140.153 25 0
30     1254722770.696634000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 192.168.1.1,10.10.1.4 1470 10.10.1.4,74.53.140.153 25 0
32     1254722771.104972000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
33     1254722771.104998000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
35     1254722771.469759000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
36     1254722771.469814000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
38     1254722771.494181000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
39     1254722771.494199000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
41     1254722771.834628000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
42     1254722771.834655000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
44     1254722771.858316000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
45     1254722771.858334000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
52     1254722772.248789000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 74.53.140.153 25 10.10.1.4 1470 0
54     1254722774.763825000 00:e0:1c:3c:17:c2 00:1f:33:d9:81:60 10.10.1.4 1470 74.53.140.153 25 0
56     1254722775.105467000 00:1f:33:d9:81:60 00:e0:1c:3c:17:c2 74.53.140.153 25 10.10.1.4 1470 0
```

Fig 14: Network Metadata details

This output shows the full network-layer metadata for the captured SMTP session, including Ethernet MAC addresses, source/destination IP addresses, TCP ports, and timestamps for each frame in the conversation between client 10.10.1.4 and mail server 74.53.140.153.



```
(kali@kali)-[~/SBT-DF203-Lab4]
$ tshark -r working/smtp_working.pcap -Y 'smtp contains "User-Agent" || smtp contains "X-Mailer"' \
  -T fields -e frame.number -e tcp.stream -e smtp.req.parameter -e data-text-lines \
  | tee reports/client_indicators.tsv
22      0      Line-based text data (47 lines)
26      0      Line-based text data (12 lines)

(kali@kali)-[~/SBT-DF203-Lab4]
$
```

Fig 15: Data Texts

Part F - Encryption and Evidential Limitations

Looking back at the capture, the server did advertise **STARTTLS** as an available capability in its EHLO response (frame 9), but the client never actually issued a **STARTTLS** command afterward. Instead, it proceeded directly with **AUTH LOGIN** in plaintext, meaning no TLS negotiation occurred at any point in this session. As a result, the entire conversation, including the username, password, and full email content, remained unencrypted and fully readable in the capture.

Since the session was never encrypted, there are no encryption-related visibility limits to report here; everything (headers, credentials, and message body) was already visible in plaintext. However, it's worth noting for completeness that if **STARTTLS** had been used, only limited metadata would remain visible afterward, source/destination IPs and ports,

packet timing, byte counts, and TLS handshake/certificate details, while the actual SMTP commands, credentials, and message content would be encrypted and unreadable without the private key.

Required Findings Worksheet

Question	Finding
When did the SMTP session start and end?	Started 2009-10-05 07:06:08.219663 and ended 2009-10-05 07:06:11+
Client IP/MAC and port	IP: 10.10.1.4, MAC: 00:e0:1c:3c:17:c2, Port: 1470
Server IP/MAC and port	IP: 74.53.140.153, MAC: 00:1f:33:d9:81:60, Port: 25
SMTP server banner	220 xc90.websitewelcome.com ESMTP Exim 4.69 #1 Mon, 05 Oct 2009 01:05:54 -0500
Client software	Microsoft Office Outlook 12.0
Authentication method	AUTH LOGIN
Envelope sender and recipient	Sender (MAIL FROM): gurpartap@patriots.in Recipient (RCPT TO): raj_deol2002in@yahoo.co.in
Message From/To/Subject	From: "Gurpartap Singh" gurpartap@patriots.in To: raj_deol2002in@yahoo.co.in Subject: SMTP
Message body type	multipart/mixed containing a multipart/alternative body (text/plain and text/html versions), plus an additional text/plain part outside the alternative block, indicating a likely attachment.
Attachment present?	Yes (a third text/plain MIME part appears outside the multipart/alternative block, consistent with an attached text file)
STARTTLS/TLS observed?	No. STARTTLS was advertised by the server (frame 9) but never invoked by the client. The entire session, including credentials and message body, remained in plaintext.
Key limitations	There was no key Limitation Since no encryption was used, this specific capture provides full visibility into commands, credentials, and message content. However, this should not be generalized: on encrypted sessions, only endpoint metadata (IPs, ports, timing, byte counts) would remain visible, and message content or credentials could not be claimed without genuine plaintext or a compromised key.